음성 AI 전문가 과정 · Day 12 / 40

Day 12

Whisper Full Fine-tuning Seq2SeqTrainer · Data Collator · 모니터링

Teacher forcing → Zeroth 학습 데이터 → Data Collator(−100) → compute_metrics(CER) → 학습 설정 · Trainer → TensorBoard 곡선 → 전 · 후 비교 · 저장

3 주차 M3 · 4 시간 · T4 GPU · 실습 노트북 Part 0 ~ 8

```
{'loss': 0.62, 'learning_rate': 1e-05, 'epoch': 0.8}
{'eval_loss': 0.41, 'eval_cer': 9.8}   # step 100
{'eval_loss': 0.35, 'eval_cer': 7.1}   # step 200
{'eval_loss': 0.36, 'eval_cer': 7.4}   # step 300

CER 학습 전 12.4% → 학습 후 7.1%
best: checkpoint-200
pipeline("automatic-speech-recognition",
         model="whisper-small-ko/final")
```

출력 모양 예시 · 숫자는 GPU · 데이터 · 무작위성에 따라 다릅니다

0.0 · 오늘 답할 질문

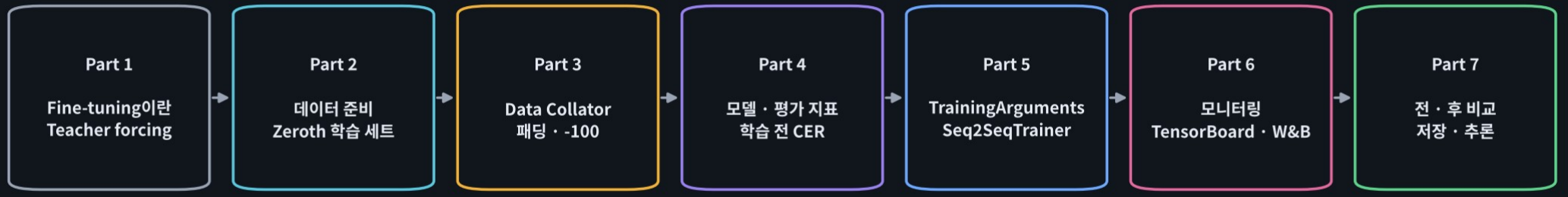
이미 똑똑한 Whisper 를 어떻게 우리 데이터로 더 가르치나

1	Fine-tuning 은 무엇이고 , Whisper 는 무엇을 맞히며 배우나 ?	Part 1
2	길이가 다른 샘플을 어떻게 한 묶음으로 만드나 ? 왜 – 100 인가 ?	Part 2 ~ 3
3	학습 중 점수 (CER) 는 어떻게 매기나 ?	Part 4
4	학습률 · 묶음 크기 · 메모리는 어떻게 정하나 ?	Part 5
5	학습 곡선에서 무엇을 읽어 내나 ?	Part 6
6	학습이 정말 도움이 됐는지 어떻게 확인하고 저장하나 ?	Part 7

▶ 애니메이션 D12-1 오늘의 흐름

개념 → 부품 → 학습 → 곡선 → 비교

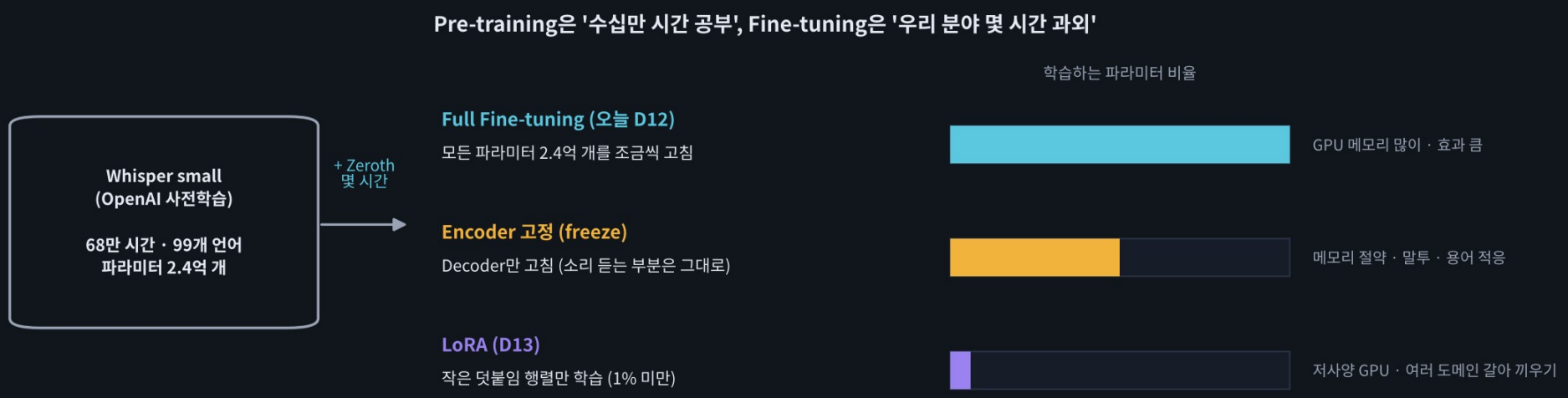
D12 · 이미 똑똑한 Whisper를 '우리 데이터'로 조금 더 가르치기



D11 데이터 준비 → D12 Full Fine-tuning → D13 LoRA → D14 비교 · 상용 API · Hub 업로드 → D15 미니 프로젝트

교시	내용	실습
1 교시	Fine-tuning 개념 , Teacher forcing, Zeroth 학습 데이터 · set_transform	Part 0 ~ 2
2 교시	Data Collator(−100), compute_metrics(CER), 학습 전 받아쓰기	Part 3 ~ 4 · 빈칸 1 ~ 5
3 교시	학습률 · 메모리 , TrainingArguments, Trainer, 🚀 학습 실행 (20~30 분)	Part 5 · 빈칸 6 ~ 8
4 교시	TensorBoard 곡선 읽기 , 전 · 후 비교 , 저장 · pipeline	Part 6 ~ 8 · 빈칸 9 · 노션

수십만 시간 공부한 모델에게 우리 분야 과외



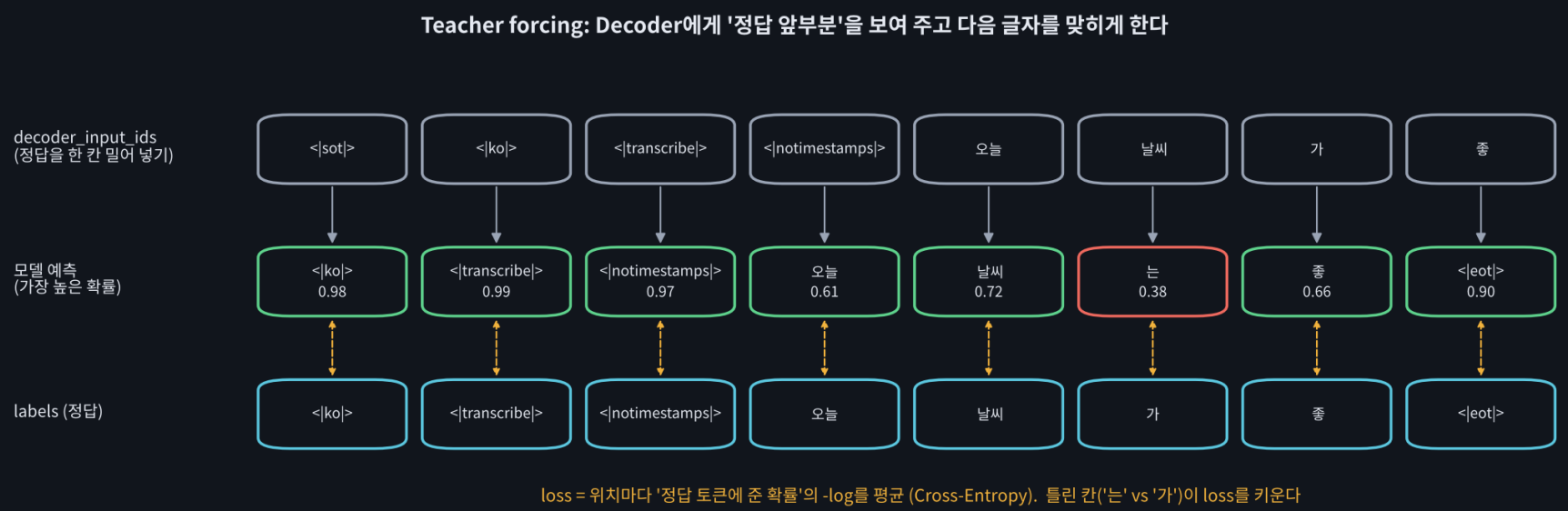
🍎 경력직 신입사원 교육

일 잘하는 경력직 (Whisper) 에게 회사 용어 · 업무 방식 (우리 데이터) 을 며칠 가르친다 . 너무 세게 가르치면 원래 알던 것을 잊는다 → 학습률을 아주 작게 .

▶ 애니메이션 D12-2 Fine-tuning 방식 비교

오늘 : whisper-small 2.4 억 개 전부 학습

Decoder 는 정답 앞부분을 보고 다음 토큰을 맞힌다



핵심 식 하나 : loss = 평균 (−log P(정답 토큰))

확률 1 → 0 · 0.5 → 0.69 · 0.1 → 2.3

▶ 애니메이션 D12-3 Teacher forcing과 loss

실제 생성 : 자기 예측이 다음 입력 → 틀리면 번짐

Zeroth 학습 세트 2,000 개 + 공식 시험 세트 100 개

```
stream = load_dataset("kresnik/zeroth_korean", split="train",
            streaming=True).cast_column("audio", HFAudio(decode=False))
train_rows, _ = collect(stream, 2000)       # 빈 전사 · 30 초 초과 제외

test_all = load_dataset("kresnik/zeroth_korean", split="test")
pick = test_all.shuffle(seed=0).select(range(200))
test_rows, _ = collect(pick, 100)
# 겹치는 화자 : 없음 ✓
```

세트	개수	쓰임
train	2,000 (≈4.6h)	학습
test	100	평가 · 비교

⚠ **평가 데이터는 학습에 절대 섞지 않기**

💬 **쉽게 말하면** D11 방식 그대로 : 바이트로 받아 soundfile 로 읽고 , 16 kHz 모노로 맞추고 , 품질 필터 .

특징은 꺼낼 때마다 만든다

```
def prepare_batch(batch):
    feats = processor.feature_extractor(batch["audio"],
                sampling_rate=SR).input_features
    labels = [processor.tokenizer(t).input_ids for t in batch["text"]]
    return {"input_features": feats, "labels": labels}
train_ds.set_transform(prepare_batch)
```

방식	계산	디스크
map + 저장 (D11)	미리 한 번	많이
set_transform	꺼낼 때마다	거의 0

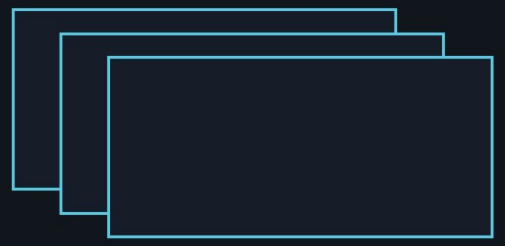
❓ remove_unused_columns=False 는 왜 ?

Trainer 는 모델 forward 가 받지 않는 열 (audio, text) 을 미리 지운다 → transform 이 받을 재료가 사라짐 → 열 지우기를 끈다 .

길이가 다른 labels 를 채우고 , 채운 칸은 – 100

Data Collator: 길이가 다른 샘플을 한 묶음(batch)으로 맞추기

input_features: 이미 모두 80 × 3000 → 그냥 쌓기



(8, 80, 3000)
batch × mel × 시간

labels: 길이가 제각각 → 가장 긴 것에 맞춰 채우고, 채운 칸은 -100

50264	50359	50363	12421	10175	9466	50257	-100
50264	50359	50363	8880	50257	-100	-100	-100
50264	50359	50363	4446	11534	1638	2195	50257

-100 = 'loss를 계산하지 마라' 표시 (PyTorch CrossEntropy의 ignore_index)
앞의 <|startoftranscript|>(50258)는 모델이 알아서 붙이므로 labels에서 뺀다

🍎 시험지 채점

문항 수가 다른 시험지를 한 묶음으로 채점기에 넣으려고 빈칸을 덧붙였다면 , 그 빈칸에는 " 채점 제외 " 도장 (–100) 을 찍는다 .

▶ 애니메이션 D12-4 Data Collator

소리는 쌓고 , 글자는 채우고 , −100, 시작 토큰 빼기

```
def __call__(self, features):
    audio = [{"input_features": f["input_features"]} for f in features]
    batch = self.processor.feature_extractor.pad(audio, return_tensors="pt")
    lab = [{"input_ids": f["labels"]} for f in features]
    lab_batch = self.processor.tokenizer.pad(lab, return_tensors="pt")
    labels = lab_batch["input_ids"].masked_fill(
                lab_batch.attention_mask.ne(1), -100)
    if (labels[:, 0] == self.decoder_start_token_id).all().cpu().item():
        labels = labels[:, 1:]
    batch["labels"] = labels; return batch
```

🧩 빈칸 실습 1

ne(1), ____ → -100

🧩 빈칸 실습 2

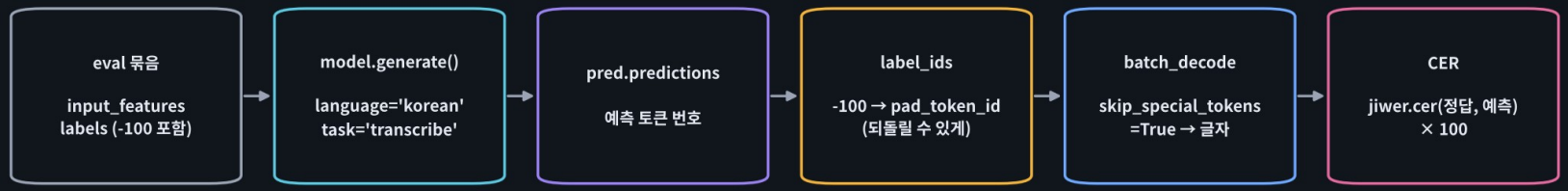
== ____ → self.decoder_start_token_id

```
{'input_features': (4, 80, 3000),
 'labels': (4, 23)}
−100 개수 : [0, 9, 8, 2]
```

▶ 애니메이션 D12-8 예측 → CER

예측 토큰 → 글자 → 정규화 → CER

평가: predict_with_generate=True → 실제로 받아쓰게 한 뒤 CER 계산 (compute_metrics)



teacher forcing으로 잰 eval loss와 달리, generate CER은 '앞에서 틀리면 뒤도 흔들리는' 실제 사용 조건

```
def compute_metrics(pred):
    label_ids = pred.label_ids.copy()
    label_ids[label_ids == -100] = processor.tokenizer.pad_token_id
    dec = lambda ids: processor.batch_decode(ids, skip_special_tokens=True)
    pred_str = [normalize(s) for s in dec(pred.predictions)]
    label_str = [normalize(s) for s in dec(label_ids)]
    return {"cer": 100 * jiwer.cer(label_str, pred_str)}
```

🧩 빈칸 실습 3

= ____ → processor.tokenizer.pad_token_id

🧩 빈칸 실습 4

jiwer.cer(____, pred_str) → label_str

한국어 받아쓰기로 고정 , 그리고 accum 함정 하나

```
model = WhisperForConditionalGeneration.from_pretrained("openai/whisper-small")
model.generation_config.language = "korean"
model.generation_config.task = "transcribe"
model.accepts_loss_kwargs = False   # accum 때 loss 평균 보장

out = model(input_features=b["input_features"], labels=b["labels"])
out.loss  # 학습 전 teacher forcing loss
```

🧩 **빈칸 실습 5**

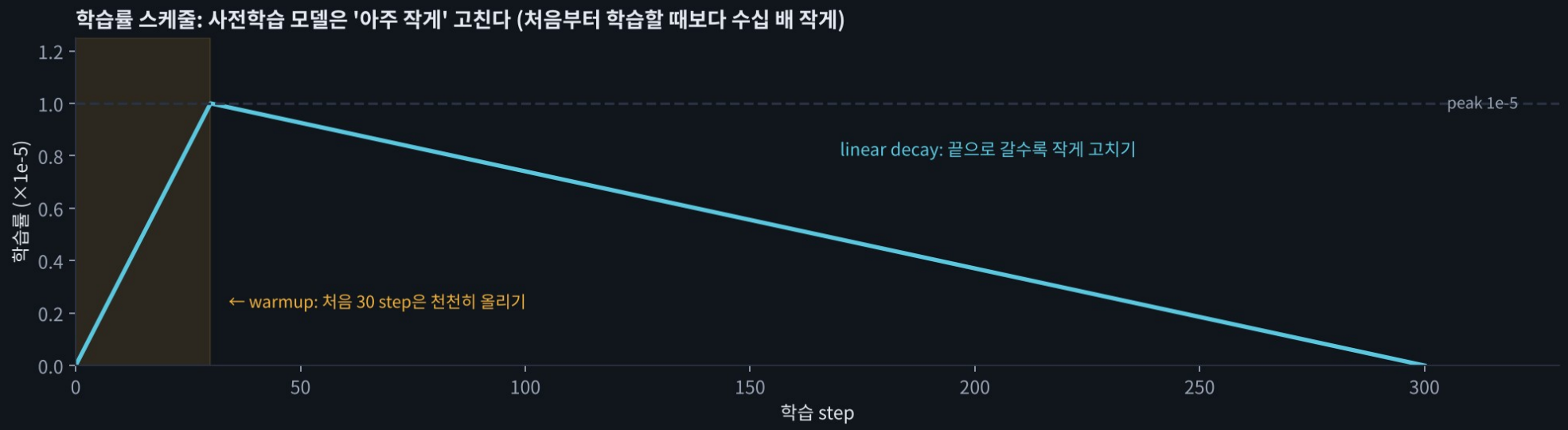
language = ____ → "korean"

accum 처리	기록 loss	실제 효과
기본 (5.x)	≈ 2 배	lr × 2
accepts_loss_kwargs=False	정상	정상

💬 **쉽게 말하면** 학습 전에 같은 문장 5 개를 받아써 두어야 학습 후 나란히 비교할 수 있다 (baseline).

```
before = transcribe(model, test_rows)   # model.generate(..., language="korean")
# 정답 : 선생님께서 오늘 수업을 일찍 마치셨습니다
# 학습 전 : 선생님께서 오늘 수업을 일찍 마치셨습니다 .
```

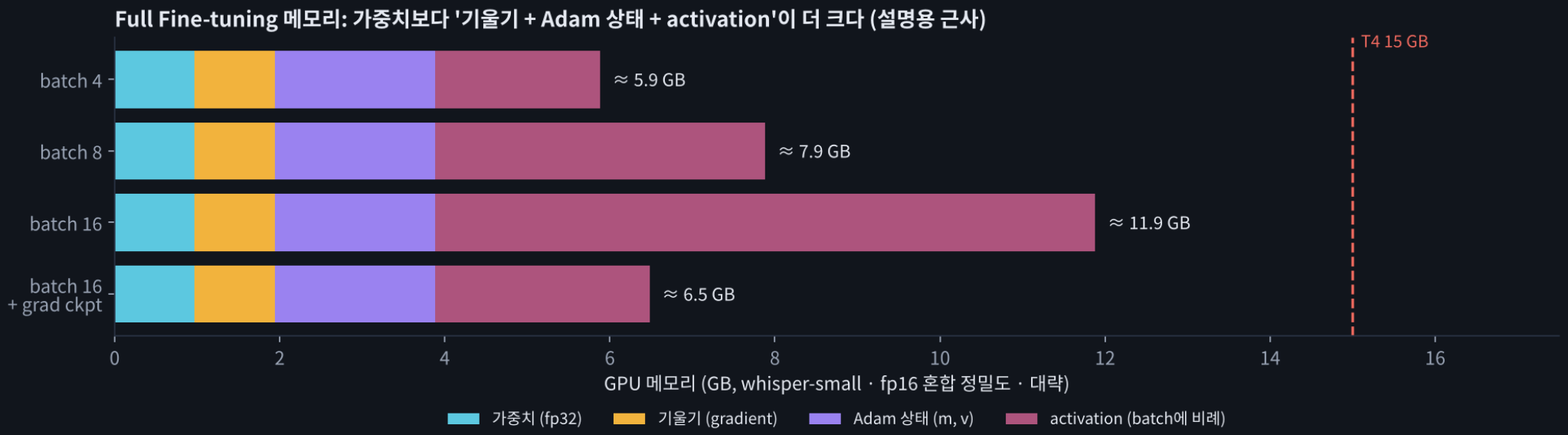
사전학습 모델은 아주 작게 고친다 : 1e-5



학습률	결과
1e-3 (처음부터 학습 수준)	loss 튐 · 원래 실력 잊음
1e-5 (오늘)	loss ↓ · 원래 실력 대부분 유지
1e-7	거의 변하지 않음

▶ 애니메이션 D12-5 학습률 실험실

가중치보다 기울기 + Adam + activation 이 크다



🍎 국을 두 번 나눠 끓이기

큰 냄비 (GPU) 가 없어 국을 두 번에 나눠 끓인 뒤 한 솥에 합쳐 간을 본다 = gradient accumulation. 맛 (학습 효과) 은 큰 냄비 한 번과 거의 같다 .

▶ 애니메이션 D12-6 GPU 메모리 계산기

학습 계획표

```
args = Seq2SeqTrainingArguments(
    output_dir="whisper-small-ko",
    per_device_train_batch_size=8, gradient_accumulation_steps=2,
    learning_rate=1e-5, warmup_steps=30, max_steps=300,
    fp16=True,
    eval_strategy="steps", eval_steps=100, save_steps=100,
    predict_with_generate=True, generation_max_length=225,
    load_best_model_at_end=True, metric_for_best_model="cer",
    greater_is_better=False, save_total_limit=2,
    remove_unused_columns=False,
    report_to=["tensorboard"])
```

🧩 빈칸 실습 6

learning_rate=____ → 1e-5

🧩 빈칸 실습 7

predict_with_generate=____ → True

값	뜻
8 × 2 = 16	실제 묶음
300 step	4,800 샘플 ≈ 2.4 바퀴
fp16	빠르고 메모리 절약

계획표대로 학습 · 평가 · 저장을 대신 돌리는 코치

```
trainer = Seq2SeqTrainer(
    model=model, args=args,
    train_dataset=train_ds, eval_dataset=test_ds,
    data_collator=collator,
    compute_metrics=compute_metrics,
    processing_class=processor)
base = trainer.evaluate()      # 학습 전 eval_cer
result = trainer.train()       # 🚀 T4 약 20~30분
```

🧩 빈칸 실습 8

data_collator=____ → collator

체크포인트: 중간 저장 → 가장 좋은 것만 남기고, 끝에 그것을 불러오기



save_steps = eval_steps · save_total_limit=2 (디스크 절약) · load_best_model_at_end=True · metric_for_best_model='cer' · greater_is_better=False

한 step 동안 일어나는 일

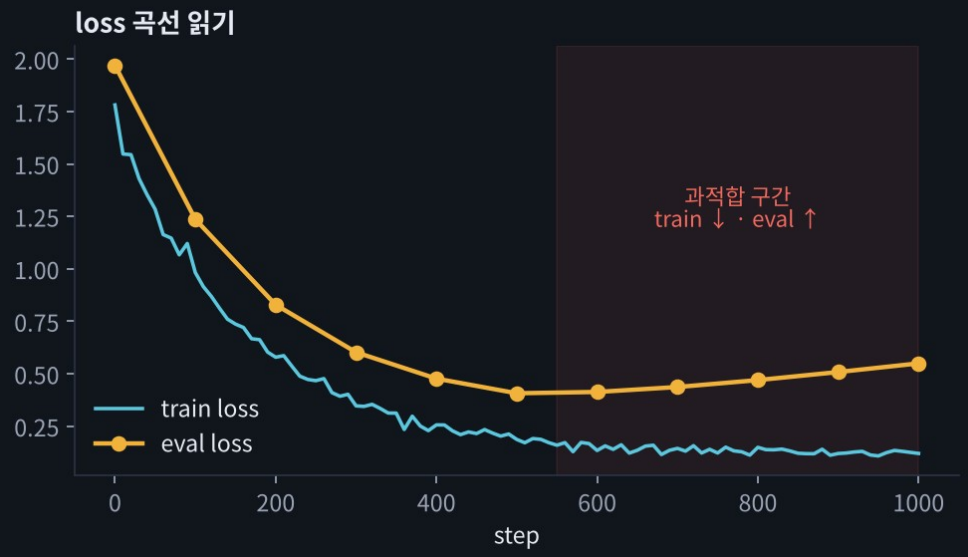
1. 묶음 8 개 꺼내기 (set_transform → 특징 · 토큰)
2. Collator: 패딩 · −100 · 텐서
3. forward (teacher forcing) → loss
4. backward → 기울기 쌓기
5. 2 번 모이면 optimizer 가 가중치를 조금 고침
6. 학습률 갱신 · 기울기 비우기
7. 100 step 마다 평가 (generate → CER) · 저장

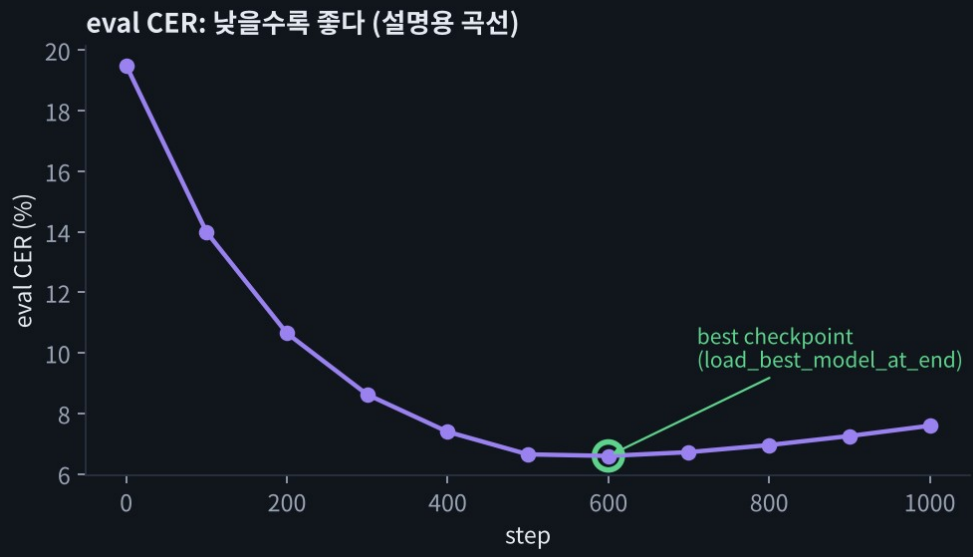
▶ 애니메이션 D12-7 학습 한 step 따라가기

⚠ CUDA out of memory

1. batch 4 × accum 4 (실제 묶음 16 유지)
2. gradient_checkpointing=True
3. model.freeze_encoder()
4. LoRA (D13)

train ↓ · eval ↑ 이면 과적합





곡선	뜻	조치
train ↓ · eval ↓	잘 배우는 중	더 학습해 볼 만함
train ↓ · eval ↑	과적합	best 체크포인트 · 데이터 늘리기
loss 튐 · NaN	학습률 너무 큼	lr ↓ · warmup ↑
거의 평평	학습률 너무 작음	lr ↑

▶ 애니메이션 D12-9 곡선 읽기 연습

Colab 안에서 곡선 보기 , 기록 직접 꺼내 그리기

```
%load_ext tensorboard
%tensorboard --logdir {CFG["out_dir"]}/runs
```

```
log = trainer.state.log_history
tr = [(h["step"], h["loss"]) for h in log if "loss" in h]
ev = [(h["step"], h["eval_loss"], h["eval_cer"])
      for h in log if "eval_cer" in h]
```

🧩 **빈칸 실습 9**

if ____ in h → "eval_cer"

도구	특징
TensorBoard	Colab 안 · 설치 쉬움
W&B	웹 대시보드 · 실험 비교 · 키 필요
log_history	파이썬으로 직접

```
# (선택) W&B
USE_WANDB = True   # 🔑 WANDB_API_KEY
report_to=["tensorboard", "wandb"]
```

💬 **쉽게 말하면** 숫자 표보다 곡선 한 장이 빠르다 .

같은 문장을 나란히 읽고 , 한 폴더에 저장

```
after = transcribe(trainer.model, test_rows)
final = trainer.evaluate()
trainer.save_model(FINAL)
processor.save_pretrained(FINAL)
asr = pipeline("automatic-speech-recognition", model=FINAL)
asr({"raw": y, "sampling_rate": 16000},
    generate_kwargs={"language": "korean", "task": "transcribe"})
```

	학습 전	학습 후
CER (예)	12.4%	7.1%
띄어쓰기	Whisper 습관	Zeroth 규칙
문장 부호	마침표 붙임	없음

❓ 뉴스 낭독으로 학습한 모델이 회의 녹음 · 영어도 더 잘할까 ?

보장 안 됨 (catastrophic forgetting). 서비스와 비슷한 데이터로 학습하고 , 원래 잘하던 일도 따로 평가 . 여러 도메인이면 LoRA(D13).

💾 Colab 세션이 끝나면 사라짐 → Google Drive 에 복사 (약 1 GB)

정리 · 다음 시간

부품 다섯 개로 Whisper 를 고쳐 썼다

- ☐ Fine-tuning 방식과 teacher forcing · loss 를 설명했다
- ☐ set_transform 으로 Zeroth 학습 데이터를 준비했다
- ☐ Collator(−100 · 시작 토큰 빼기) 를 만들었다
- ☐ compute_metrics 로 학습 전 CER 을 쟀다
- ☐ TrainingArguments · Trainer 로 학습을 돌렸다
- ☐ 곡선을 읽고 전 · 후를 비교해 저장했다

다음 : D13 PEFT / LoRA Fine-tuning

2.4 억 개를 다 고치지 않고 1% 미만의 덧붙임 행렬만 학습합니다 . 8-bit 로딩과 함께 저사양 GPU 에서 더 큰 모델을 다루고 , 의료 · 법률 · 산업 도메인 용어에 특화시켜 봅니다 . 오늘의 Collator · compute_metrics · Trainer 를 그대로 씁니다 .

worksheet.html	40 문항
개념 워크시트 · 노션 제출	
animation.html	9 개 탭
인터랙티브 애니메이션	
D12_Whisper_FullFinetuning_실습 .ipynb	빈칸 9
실습 노트북 · T4	

온라인 자료 imguru-mooc.github.io/AI_Voice/day12